Program: BE

Date	17.12.2025	Maximum Marks	65
Course Code	CS235AI	Duration	120 Min
Sem	III	CIE – II	

OPERATING SYSTEMS
(Common to CSE/CD/CY/ISE)
Scheme and Solution

Sl. No.	Quiz Questions	M	L	CO
1	Because one long process at the front delays all shorter processes behind it, increasing waiting and turnaround time.	**2**	L2	CO2
2	The main drawbacks of spinlocks are that they waste CPU cycles through busy-waiting, are not power-efficient, can lead to priority inversion and starvation, and are unsuitable for long-held locks or when the lock holder might be preempted. This contrasts with semaphores, which put a thread to sleep instead of busy-waiting when the lock is unavailable.	**2**	L2	CO2
3	Only the first reader blocks writers by waiting on wrt. Remaining readers can enter freely without reblocking the writer semaphore unnecessarily.	**2**	L3	CO2
4	Binary and counting semaphore with example -1m+1m	**2**	L1	CO1
5	Initial = 3, After ops = 3 → 2 → 1 → 2 → 1 **Final value = 1**	**2**	L3	CO4

Sl. No.	Test Questions	M	L	CO
1	Preemptive SJF, Preemptive Priority and Round Robin Gantt charts(1+1+2)=4m TT & WT (1+1+2)=4m No of context switches = 2m ALGORITHM | AWT | ATAT | No of Context switches Preemptive SJF | 11.8 | 25.2 | 5 Preemptive Priority | 29 | 42.5 | 6 Round Robin | 23.2 | 36.6 | 28	**10**	L3	CO3
2a	(06m) CPU utilization. Throughput Turnaround time Waiting time Response time	**6**	L2	CO2

2b	(2m) Two process solution which assumes that the **load** and **store** machine-language instructions are atomic. Two processes share two variables: **int turn;** **Boolean flag[2]** The variable **turn** indicates whose turn it is to enter the critical section The **flag** array is used to indicate if a process is ready to enter the critical section. **flag[i] = *true*** implies that process $\mathbf{P_i}$ is ready! (2m) **do {** **flag[i] = true;** **turn = j;** **while (flag[j] && turn = = j);** **critical section** **flag[i] = false;** **remainder section** **} while (true);**	**4**	L1	CO1
3a	TestAndSet() - 3m **boolean test_and_set (boolean *target)** **{** **boolean rv = *target;** ***target = TRUE;** **return rv:** **}** Executed atomically Returns the original value of passed parameter Set the new value of passed parameter to "TRUE". Shared Boolean variable lock, initialized to FALSE Solution: **do {** **while (test_and_set(&lock))** **; /* do nothing */** **/* critical section */** **lock = false;** **/* remainder section */** **} while (true);** CompareAndSwap() - 3m **int compare _and_swap(int *value, int expected, int new_value) {** **int temp = *value;** **if (*value == expected)** ***value = new_value;** **return temp;** **}** Executed atomically Returns the original value of passed parameter "value" Set the variable "value" the value of the passed parameter "new_value" but only if "value" =="expected". That is, the swap takes place only	**6**	L3	CO4

	under this condition. Shared integer "lock" initialized to 0; Solution: **do {** **while (compare_and_swap(&lock, 0, 1) != 0)** **; /* do nothing */** **/* critical section */** **lock = 0;** **/* remainder section */** **} while (true);**			

3b	The structure of a reader process
	do
	{
	wait(mutex);
	read_count++;
	if (read_count == 1)
	wait(rw_mutex);
	signal(mutex);
	...
	/* reading is performed */
	...
	wait(mutex);
	readcount--;
	if (read_count == 0)
	signal(rw_mutex);
	signal(mutex);
	} while (true);
		4	L3	CO3

4a	General structure – 2m Diagram – 1m Requirements – 4m	**7**	L2	CO1
4b		**3**	L2	CO2

	Feature	**Preemptive Scheduling**	**Non-Preemptive Scheduling**			
	Process Interruption	A running process can be interrupted by another process, often if a higher-priority one arrives.	A process cannot be interrupted and must complete its execution or enter a waiting state before another process can run.			
	CPU Allocation	Allocated for a fixed amount of time (time quantum).	Allocated until the process finishes its task or becomes I/O-bound.			
	Overhead	Higher overhead due to context switching between processes.	Lower overhead as there is no context switching between processes.			
	Flexibility	More flexible, allowing critical processes to run immediately.	Less flexible, as it does not allow a higher-priority process to preempt a running one.			
	Starvation	Can lead to starvation for lower-priority processes if higher-priority ones arrive frequently.	Can lead to starvation for processes with short burst times if a long process is running.			
	Examples	Round Robin, Shortest Remaining Time First (SRTF), Preemptive Priority.	First-Come, First-Served (FCFS), Shortest Job First (SJF), Non-preemptive Priority.			
5	Race condition with example – 2m Code for Producer thread – 3m Code for consumer thread – 3m Complete program calling threads -2m			**10**	L3	CO5

Course Outcomes

CO 1	Demonstrate the fundamental concepts of operating system like process management, file management, memory management and issues of synchronization.
CO 2	Analyze and interpret operating system concepts to acquire a detailed understanding of the course.
CO 3	Apply the operating systems concepts to address related new problems in computer science Domain.
CO 4	Design or develop solutions to solve applicable problems in operating systems domain.
CO5	Extend the theoretical knowledge acquired through the course to demonstrate skills like investigation, effective communication, working in team/Individual, following ethical practices by implementing operating system concepts/applications and engage in lifelong learning.

Blooms' taxonomy

L1	L2	L3	L4	L5	L6	CO1	CO2	CO3	CO4	CO5	CO6
06	20	34				13	15	14	8	10	